

Akamai Edge Redirector DevOps Integration

- [Overview](#)
- [Strategic Goals](#)
- [Current State vs. Future State](#)
- [Functional Use Cases](#)
- [Edge Case & Exception Logic](#)
- [Technical Workflow & CI/CD Architecture](#)
- [Governance & RBAC](#)
- [Rollback & Disaster Recovery](#)

Overview

We are aiming to modernize our Akamai Edge infrastructure by transitioning from manual portal management to a GitHub/Terraform-driven DevOps workflow. This initiative replaces the labor-intensive manual handling of over 50,000 redirects currently handled by the DEAP team with a streamlined, version-controlled process.

Strategic Goals

- **Automation & Speed:** Replace manual UI entries with CI/CD pipelines to handle the weekly volume of 5–10 new requests and complex bulk updates.
- **Self-Service Empowerment:** Enable teams (FDC, B2B, and BigW Genesis) to own their redirect configurations through version-controlled templates.
- **Risk Reduction:** Implement automated "Fail-Fast" validations to catch circular loops, duplicate entries and high-risk "catch-all" redirects before they reach production.
- **Integration Support:** Partner with Akamai Professional Services over a six-week period to customize and deploy a robust, scalable redirect modules.

Current State vs. Future State

Feature	Current State (Manual)	Future State (DevOps)
Interface	Akamai Control Center (Manual UI)	GitHub/Terraform, (JSON/CSV) & CI/CD Pipelines
Execution	Manual entry per URL	API-based automated deployment
Validation	Human review / Basic portal check	Automated pre-deployment linting & API validation
Storage	Cloudlet Policy Versions	Version-controlled GitHub Repo

Functional Use Cases

UC ID	Use Case Name	Description	Key Inputs / Parameters	Validation & Business Logic
UC-01	Standard Single Redirect	Addition or update of a 1:1 URL mapping.	Start Date/Time: End Date/Time: Source URL: Target URL: Redirect Code: 301/302. Relative URL validation: Copy Query String:	• Check for existing entry. • Ensure target is not the same as source. • If target is used as source of existing redirect, highlight it
UC-02	Bulk Redirect Upload	Addition or update of Bulk URL redirects	Start Date/Time: End Date/Time: Source URL: Target URL: Redirect Code: 301/302. Relative URL validation: Copy Query String:	• Check for existing entry. • Ensure target is not the same as source. • If target is used as source of existing redirect, highlight it
UC-03	Regular Expression Redirect	Addition or update of a 1:1 Regex URL mapping.	Regular Expression (Regex): Redirect URL:	• Check for existing entry. • Ensure target is not the same as source.

			Show Regex Tester: Test URL: Redirect Code: 301/302. Relative URL validation: Copy Query String:	• If target is used as source of existing redirect, highlight it
--	--	--	--	--

Edge Case & Exception Logic

The following technical exceptions must be caught by the CI/CD pipeline "Validator" before the API call is executed.

Ref ID	Scenario / Risk	Technical Logic / Rule	Pipeline Action (CI/CD)
EC-01	Circular Redirect (Loop)	Detects if URL A → URL B and a new rule creates URL B → URL A .	REJECT: Throw error "Potential Redirect Loop Detected."
EC-02	Wild card Redirect	Detects if * char present in the source/destination URL path	WARN: Throw error "Potential Content loading Issue Detected due to wild card pattern" Additional approval required to proceed the deployment
EC-03	Entire Hostname redirect	Detects if /* present in the source URL	WARN: Throw error "Potential Entire Website will be Impacted"

			Additional approval required to proceed the deployment
EC-03	Homepage redirect	Detects if / alone present in the source URL	WARN: Throw error "Home page will not be Loaded" Additional approval required to proceed the deployment

Technical Workflow & CI/CD Architecture

To Do:

- ☐ Add Workflow diagram
- ☐ Add CI/CD Architecture Diagram

Governance & RBAC

To maintain security while decentralizing management, the following permissions are enforced:

Role	Responsibility	Permissions

Rollback & Disaster Recovery

In the event of a production issue (e.g., unintended traffic loss), the following recovery steps must be followed:

- GitHub Revert:
- Emergency Manual Override:
- SLA for Recovery: